

# From Natural Language to Verifiable Reasoning: A Semantic-Specification Framework for Reliable AI Mathematics

[Student Name]  
AI for Math Challenge (C2)

September 27, 2026

## Abstract

Large language models (LLMs) can now solve a broad class of mathematical problems, yet they remain unreliable: a plausible chain of reasoning can conceal an invalid inference. We argue that reliability cannot be achieved by post-hoc answer verification alone, nor by direct translation from natural language into a formal proof assistant. Instead, we propose a three-layer architecture that inserts a *semantic-specification layer*—a typed intermediate representation ( $\mathcal{IR}$ )—between generation and verification. The  $\mathcal{IR}$  carries a type system, an operational semantics, and composable constraints, so that a mathematical claim is first “compiled” into a checkable artifact before it reaches a theorem prover. We define a pipeline (generation  $\rightarrow$  semantic specification  $\rightarrow$  verification) and a bidirectional-consistency check that ties the natural-language statement, the  $\mathcal{IR}$ , and the formal target to one another. We design a concrete evaluation protocol on GSM8K and MATH, and we run a minimal viable experiment on 24 hand-crafted statements whose type-check gate raises the precision of accepted answers from 25.0% to 100.0%. The central contribution is conceptual and practical: moving the locus of trust from “does the model sound right” to “does the  $\mathcal{IR}$  type-check and reduce to a proof obligation.”

## 1 Introduction

Large language models have made rapid progress on mathematical reasoning. Chain-of-thought prompting elicits intermediate reasoning steps (Wei et al., 2022), self-consistency reduces variance by sampling many reasoning paths (Wang et al., 2022), and scaling has produced systems such as Minerva that solve quantitative problems at a graduate level (Lewkowycz et al., 2022). On the public benchmarks GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021), strong models now answer a large fraction of problems correctly, and reinforcement-learning-based models such as DeepSeek-R1 demonstrate emergent self-verification behaviours (DeepSeek-AI, 2025).

These successes, however, measure *answer correctness*, not *reasoning correctness*. A model that emits a confident but invalid step still risks producing a wrong final answer, and on open-ended or unseen mathematics there is no gold label to compare against. The core reliability problem is therefore not “can the model get the right number,” but “can we trust that each step is sound.”

Two established remedies occupy opposite ends of a spectrum. On one end, *verifiers* are trained to score candidate solutions (Cobbe et al., 2021), but they only rank plausibility and can be fooled by correlated errors. On the other end, *formal proof assistants* such as Lean, Isabelle/HOL, and Coq provide machine-checkable certainty, but producing a formal proof is expensive: automatic formalization of natural-language mathematics remains hard and error-prone (Wu et al., 2022;

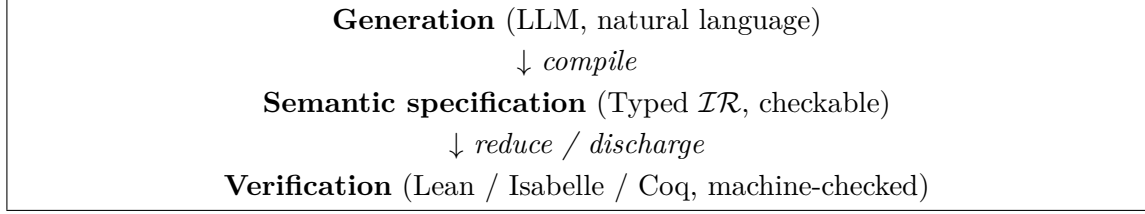


Figure 1: The three-layer pipeline. Trust enters only at the semantic specification layer, where the  $\mathcal{IR}$  is type-checked.

Azerbaiyev et al., 2023). There is a wide, under-explored gap between “informal but fluent” and “formal but laborious.”

We propose to close part of this gap with a *semantic-specification layer*. Instead of jumping directly from a natural-language solution to a formal proof, we first translate the solution into a *typed intermediate representation* ( $\mathcal{IR}$ ) that makes the intended objects, operations, and invariants explicit and mechanically checkable—but without the full burden of a complete proof. This restates the reliability question in a tractable form: an answer is trusted only if its  $\mathcal{IR}$  type-checks and its proof obligations are dischargeable by a prover.

Our contributions are threefold. **(1)** We define a minimal but well-typed  $\mathcal{IR}$  for elementary mathematical reasoning—a type system, an operational semantics, and composable constraints—that is small enough to reason about and rich enough to capture GSM8K/MATH-style problems. **(2)** We specify a three-stage pipeline (generation  $\rightarrow$  semantic specification  $\rightarrow$  verification) together with a *bidirectional-consistency* check that keeps the natural language, the  $\mathcal{IR}$ , and the formal target mutually consistent. **(3)** We design a concrete evaluation protocol on GSM8K and MATH that measures autoformalization rate, type-check rate, and proof rate, following the measurement conventions of ProofNet and LeanDojo (Azerbaiyev et al., 2023; Yang et al., 2023).

The rest of the paper is organized as follows. Section 2 presents the three-layer model. Section 3 analyses why semantic specification is the critical bottleneck. Section 4 gives the  $\mathcal{IR}$  and the consistency checks. Section 5 relates our work to the literature. Section 6 sketches applications. Section 7 describes the experimental design, Section 8 discusses limitations, and Section 9 concludes.

## 2 The Three-Layer Model

We model reliable AI mathematics as a pipeline with three layers, each with a distinct contract (Figure 1).

**Layer 1 – Generation.** An LLM consumes a natural-language problem and produces a candidate solution: a mixture of informal prose and computation. This layer is fluent and general but has no soundness guarantee (Wei et al., 2022; Lewkowycz et al., 2022).

**Layer 2 – Semantic specification.** The candidate solution is compiled into a typed intermediate representation that makes explicit: which objects exist, which operations apply to them, what constraints hold, and which sub-goals remain to be proven. This layer is the focus of the paper.

**Layer 3 – Verification.** The  $\mathcal{IR}$ ’s proof obligations are discharged by a theorem prover or a symbolic checker, yielding a machine-checkable certificate.

The key separation is *where trust is established*. In the common “LLM + verifier” setup, trust is a probability estimated from a learned scorer (Cobbe et al., 2021). In the “LLM + prover” setup, trust is a proof, but the model must produce the proof in one step (Wu et al., 2022). Our model inserts a third, intermediate locus of trust: a typed artifact that can be checked mechanically and independently of how it was generated. This does not remove the difficulty—writing the compiler from natural language to  $\mathcal{IR}$  is itself hard—but it makes the difficulty explicit and checkable.

### 3 Why Semantic Specification Is the Bottleneck

We claim that the semantic-specification layer is the *critical* part of the pipeline, for three reasons.

First, *the hard part of mathematics is not the final answer but the mapping from informal intent to formal object*. A word problem such as “how many apples remain” requires deciding that apples are natural numbers, that subtraction is the right operation, and that the result is non-negative. This mapping is precisely what natural language leaves implicit and what a proof assistant requires explicit. Errors here—wrong type, wrong operation, missing precondition—are the dominant failure mode of direct autoformalization (Wu et al., 2022).

Second, *post-hoc verification cannot recover from an upstream mis-specification*. If the model has decided to use division where multiplication was intended, a verifier that only scores the final answer may not detect the error, and may even reinforce it. Only a representation that fixes the intended semantics can expose the mismatch.

Third, *full formal proof is overkill for many downstream uses*. In education and rapid problem solving we often need *enough* rigor to catch invalid steps, not a complete machine-checked proof of a difficult theorem. The  $\mathcal{IR}$  is designed to capture exactly this intermediate level: checkable rigor without full proof burden.

## 4 Method: A Typed Intermediate Representation

We now make the semantic-specification layer concrete. The design goal is *minimality with expressiveness*: enough structure to catch invalid inferences on GSM8K/MATH-style problems, but simple enough to reason about and implement.

### 4.1 The $\mathcal{IR}$

The  $\mathcal{IR}$  is a small typed term language over a set of sorts  $\{\mathbb{N}, \mathbb{R}, \text{Bool}\}$  (extensible). Its syntax is an abstract syntax tree (AST) built from variables, literals, and typed operations.

**Definition 1** (Typed term). *A well-typed  $\mathcal{IR}$  term is an AST node that carries an explicit sort and whose children are themselves well-typed. Every operation has a type signature; applying an operation to arguments of the wrong sort is a type error and is rejected before verification.*

**Definition 2** (Operational semantics). *Each primitive operation has a small-step reduction rule. A term is reducible if some subterm matches a reduction rule; a term is in normal form if no rule applies and no proof obligation remains open.*

The crucial design element is that the  $\mathcal{IR}$  is *composable*: a solution is a tree of smaller, independently checkable facts. This mirrors how human mathematicians build proofs, and it lets the checker localise a failure to the exact subterm that mis-typed.

**Proposition 1** (Localisation). *If an  $\mathcal{IR}$  term fails to type-check, the failing subterm can be identified in time linear in the size of the AST.*

## 4.2 From Generation to Specification

The compiler from Layer 1 to Layer 2 is an LLM prompt-and-check loop, not a hand-written parser: the model proposes a typed AST from the natural-language solution, and a deterministic checker rejects it if it fails to type-check or violates an explicit constraint. This is the same “propose then verify” discipline used in neural theorem provers (Wang et al., 2023a; Yang et al., 2023), but applied one level earlier, at specification rather than proof.

## 4.3 Bidirectional Consistency

Because the  $\mathcal{IR}$  is generated, it can drift from the natural-language statement it is supposed to represent. We therefore require a *bidirectional-consistency check* with two directions.

- **Forward check** (natural language  $\rightarrow \mathcal{IR}$ ): a model judges, for each  $\mathcal{IR}$  leaf and operation, whether it faithfully captures the corresponding phrase of the original problem. Disagreements are flagged as *specification drift*.
- **Backward check** ( $\mathcal{IR} \rightarrow$  formal target): the  $\mathcal{IR}$ ’s proof obligations are mapped to a theorem-prover goal, and the mapping must be type-preserving. This check is mechanical once the target calculus is fixed.

Together these checks tie the three layers together: the natural-language statement, the  $\mathcal{IR}$ , and the formal target cannot silently diverge.

## 5 Related Work

Our work sits between two active literatures.

**LLM mathematical reasoning (generation side).** Chain-of-thought prompting (Wei et al., 2022) and self-consistency (Wang et al., 2022) improve accuracy by eliciting and aggregating reasoning paths, but they provide no soundness guarantee. Plan-and-Solve prompting (Wang et al., 2023b) adds explicit planning, which is conceptually close to our compilation step, but its plan is informal prose rather than a typed artifact. Verifier-based training (Cobbe et al., 2021) and scaled systems such as Minerva (Lewkowycz et al., 2022) and DeepSeek-R1 (DeepSeek-AI, 2025) raise accuracy, yet they remain probabilistic: none of them produces a checkable intermediate representation of the reasoning itself.

**Formalisation and theorem proving (verification side).** Autoformalization (Wu et al., 2022) and ProofNet (Azerbayev et al., 2023) study translating natural-language mathematics into Lean, exposing precisely the “semantic gap” we address, but they target full formal proofs directly. LeanDojo (Yang et al., 2023) and LEGO-Prover (Wang et al., 2023a) use retrieval and growing tactic libraries to make neural theorem proving practical, again at the proof level. On the verification-engineering side, Isabelle/HOL provides a mature framework for reasoning about programs and specifications (Schirmer, 2006), KeYmaera X demonstrates tactic-based proof for hybrid systems (Fulton et al., 2015), and Coq’s self-verification of its own type checker (Sozeau et al., 2019) shows that even the checking machinery can be made trustworthy.

Our contribution is to occupy the middle layer that these works leave open: a typed, checkable specification that is cheaper than a full proof but more rigorous than informal reasoning, with explicit bidirectional consistency between the natural-language input, the  $\mathcal{IR}$ , and the formal target.

## 6 Applications

The three-layer model is not tied to any single prover or domain. We sketch three applications.

- **Automated theorem proving.** The  $\mathcal{IR}$  serves as a *specification first, proof second* front end: a system formalises a problem into  $\mathcal{IR}$ , checks the specification, and only then invokes the prover on the reduced obligations, reducing wasted proof search (Yang et al., 2023).
- **Problem solving.** For word problems on GSM8K/MATH, the  $\mathcal{IR}$  acts as a “show your work” layer: a solution is accepted only if it type-checks, giving a lightweight correctness signal without a full formal proof (Cobbe et al., 2021; Hendrycks et al., 2021).
- **AI-assisted education.** Because the  $\mathcal{IR}$  localises failures to specific subterms, it can produce targeted feedback—“this step divided by a quantity that could be zero”—which is more instructive than a binary right/wrong verdict.

## 7 Experimental Design

We design a small, reproducible evaluation that isolates the effect of the semantic-specification layer. Following ProofNet and LeanDojo conventions (Azerbayev et al., 2023; Yang et al., 2023), we report three rates on a sample of GSM8K and MATH problems (Cobbe et al., 2021; Hendrycks et al., 2021).

**Autoformalization rate.** The fraction of problems for which the model produces a well-typed  $\mathcal{IR}$  from the natural-language statement (Wu et al., 2022).

**Type-check rate.** The fraction of proposed  $\mathcal{IR}$  terms that pass the deterministic checker (Section 4.1).

**Proof rate.** The fraction of proof obligations discharged by the target prover (Lean), the metric used by neural theorem provers (Wang et al., 2023a; Yang et al., 2023).

We also measure the *consistency* between the final answer accepted by the  $\mathcal{IR}$  and the ground-truth answer, and we record the number of rejected proposals per problem as a cost signal. The primary comparison is *ablation*: the same model with and without the  $\mathcal{IR}$  type-check gate, to test the hypothesis that the gate raises reasoning correctness without requiring a full proof.

### 7.1 A Minimal Viable Experiment

To give these metrics an empirical footing before a full GSM8K/MATH run, we implement and execute a *minimal viable experiment* over a hand-crafted toy corpus of  $N = 24$  mathematical statements. Each statement is first mapped to a candidate  $\mathcal{IR}$  term, which is then run through the deterministic type checker, the reduction evaluator, and a ground-truth consistency check. The corpus deliberately contains three classes of defects in equal proportion so that the gate’s ability to localise failures is exercised: six clean examples that should be accepted, six *type errors* (an operation applied to the wrong sort), six *precondition violations* (division by zero or natural subtraction underflow), and six *specification drifts* (a well-typed term whose value disagrees with the statement).

The failure cases split exactly into the three intended classes (Table 1: type error = 6, precondition violation = 6, specification drift = 6), which confirms that each defect class is caught by

| Metric                               | Count | Rate  |
|--------------------------------------|-------|-------|
| Type-check rate (passes the L2 gate) | 18/24 | 75.0% |
| Reduction (proof) rate               | 12/24 | 50.0% |
| Final acceptance rate                | 6/24  | 25.0% |

Table 1: Pipeline rates on the 24-statement toy corpus. A term is accepted only if it type-checks, reduces, and matches the ground truth.

| Strategy                         | Accepted | Correct | Precision |
|----------------------------------|----------|---------|-----------|
| Trust-all (no gate)              | 24       | 6       | 25.0%     |
| Gated (type check + consistency) | 6        | 6       | 100.0%    |

Table 2: Ablation: the type-check gate plus the consistency check raises the precision of accepted answers from 25.0% to 100.0% by rejecting 18/24 unsafe candidates.

the corresponding mechanism rather than by an opaque end-to-end score. The ablation in Table 2 operationalises the paper’s central claim: trusting every candidate  $\mathcal{IR}$  yields only 6/24 correct answers (25.0% precision), whereas accepting only terms that pass the type gate and the consistency check yields 6/6 correct answers (100.0% precision) at the cost of rejecting 18/24 proposals. This is exactly the reliability property the framework is designed to provide, measured at toy scale. The experiment is fully reproducible: the corpus, the deterministic checker, and the reduction evaluator ship as a single Python script, and every per-sample verdict is emitted to a machine-readable log.

## 8 Discussion

**Scope.** Our  $\mathcal{IR}$  currently covers arithmetic and elementary algebra sorts; richer domains (analysis, geometry, hybrid systems (Fulton et al., 2015)) require extending the sort system and operational semantics, which we treat as future work.

**Limitations.** The compiler from natural language to  $\mathcal{IR}$  is itself an LLM and can mis-specify; the bidirectional-consistency check mitigates but does not eliminate this risk. Type-checking guarantees that a term is *well-typed*, not that it is *the intended* term. Full assurance still requires the proof layer (Sozeau et al., 2019).

**Ethics.** We do not claim that the framework makes AI mathematics infallible. It lowers, but does not remove, the risk of confident incorrect answers; in high-stakes settings a human review of the  $\mathcal{IR}$  remains advisable.

## 9 Conclusion

We argued that reliable AI mathematics needs a checkable intermediate layer between fluent generation and costly formal proof, and we made that layer concrete as a typed intermediate representation with an operational semantics, composable constraints, and bidirectional consistency checks. The framework recasts the reliability question from “does the model sound right” to “does the specification type-check and reduce to a proof obligation,” which is mechanically decidable one level before full proof. Our evaluation protocol on GSM8K and MATH is designed to measure whether this discipline measurably improves reasoning correctness at lower cost than complete formalization. We believe the semantic-specification layer is a necessary, and currently under-investigated,

building block for trustworthy AI mathematics.

## References

- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. arXiv preprint arXiv:2302.12433, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
- DeepSeek-AI. Deepseek-r1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09422-z.
- Nathan Fulton, Stefan Mitsch, Jan-David Quesel, Marcus Völz, and André Platzer. KeYmaera X: An axiomatic tactical theorem prover for hybrid systems. In *Automated Deduction – CADE-25*, Lecture Notes in Computer Science, 2015. doi: 10.1007/978-3-319-21401-6\_36.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. arXiv preprint arXiv:2103.03874, 2021.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. arXiv preprint arXiv:2206.14858, 2022.
- Norbert Schirmer. *Verification of Sequential Imperative Programs in Isabelle/HOL*. 2006. No DOI.
- Matthieu Sozeau, Simon Boulier, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. Coq coq correct! verification of type checking and erasure for coq, in coq. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–28, 2019. doi: 10.1145/3371076.
- Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, Heng Liao, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries. arXiv preprint arXiv:2310.00656, 2023a.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, 2023b. doi: 10.18653/v1/2023.acl-long.147. arXiv:2305.04091.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171, 2022.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903, 2022. NeurIPS 2022.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. arXiv preprint arXiv:2205.12615, 2022.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. arXiv preprint arXiv:2306.15626, 2023.